Quantum Oracle Sketching on Bank-Held Data

Limitations Found, Candidate Applications, and Next Steps

Project Report – October 2026

Purpose: Summarize about one week of focused testing (October 14–21, 2026) of Quantum Oracle Sketching (QOS) [1] on public financial data: what limits it, which bank data and problems could still be worth testing, which other methods to consider, and what to do next. Each question a reader may ask has a section that answers it. Every number is reproducible from the project repository [2] at tag `v1-qos-findings`.

Bottom line. On the data tested so far (CFPB complaint text, UCI Bank Marketing, and PaySim analytically), QOS's proven advantage does **not** translate into a practical one. The paper's guarantee needs data that is sparse in both rows and columns; these datasets are not, and the resulting cost stays astronomical even after correcting for how loose the proof is. Classical streaming learners solve the same tasks with 16–32 KiB of memory, which caps what QOS could save. These are limitations found in one week on public data and small simulations, not a disproof of the paper. Bank data with bounded row *and* column sparsity, or genuinely unstorable streams, remains untested and is listed under Candidate Bank Data and Next Steps.

How to Use This Document

If the question is...	See
What is QOS, and what does the paper require?	Background
What limitations were found, and how strong is the evidence?	Key Limitations, Findings 1–5
What was *not* tested?	Scope
Which bank data and problems could still suit QOS?	Candidate Bank Data
What other methods should be considered?	Alternative Methods
What should happen next?	Next Steps
Common objections ("it's a Caltech/Google paper", "a bigger machine", ...)	Anticipated Questions
How do I check or rerun the results?	Reproducibility

Evidence labels

Every result carries one label, so that measured facts are never confused with estimates: [measured] real data or real classical training runs; [simulated] classical simulation of QOS building blocks; [calculated] the paper's formulas with measured inputs, orders of magnitude only; [estimated] back-of-envelope estimates or extrapolations. Nothing here is a measurement of quantum hardware.

Key Limitations Found

1. **Real financial data is not sparse enough.** The theorem needs every row *and every column* of the data matrix to have few nonzeros. In complaint text, common words appear in 58–71% of documents; in tabular data, numeric columns are filled in every row. Column sparsity grows with the dataset instead of staying small. [measured]
2. **The paper's cost bound is astronomical on this data:** 10^{37}–10^{45} passes over the training data, and still 10^{11}–10^{15} with perfectly sparse data. [calculated]
3. **The bound is loose, but not loose enough.** In simulation, the most expensive building block scales like $s^{3.1}$–$s^{3.3}$ in sparsity, not s^5. Correcting for this removes 6–8 orders of magnitude and leaves 10^{30}–10^{37}. [simulated, estimated]
4. **For stored data, the standard quantum route is far cheaper.** QROM loading costs about 10^{10}–10^{11} gates per query. QOS needs about 10^{24}–10^{25} samples per query, i.e. re-streaming the dataset about 4×10^{18} times, so the data must be stored anyway. [estimated]
5. **Classical streaming learners need only kilobytes.** They reach 95% of full-model accuracy with 16–32 KiB on held-out test data, usually in one pass. The 8.2 MB text featurizer, needed by any learner, outweighs the learner itself. [measured]

Consequence. On these datasets, even a cost-free QOS could save at most the 16–32 KiB of classical learner state needed for 95% retention.

Background: QOS and What the Theorem Requires

The idea, in plain English

- Quantum machine-learning algorithms usually assume the data can be queried "in superposition" (an *oracle*) [4]. Building that access normally means storing the dataset in quantum-accessible memory, which is expensive.
- QOS streams random samples instead. Each sample nudges the quantum state by a small rotation and is thrown away; after enough samples, the rotations add up to the oracle. In the simplest case, $O_f|x\rangle = (-1)^{f(x)}|x\rangle$ is built from many phases of size $\pi f(x)/(\text{expected visits to } x)$.
- The sketched oracles feed a quantum linear solver (QSVT [5, 6]) that trains a ridge classifier, read out with classical shadows [7].
- **Claimed advantage: memory, not time.** A polylogarithmic-size quantum machine classifies where any classical machine needs nearly linear size [1]. The "about 50–60 logical qubits" in the paper's real-data examples (Eq. A9) cover a **single prediction** and exclude the costs below.

Conditions and cost (Theorem F.13 of [1])

- **Sparsity s:** most nonzeros in any row *or column*. **Conditioning κ_{reg}:** how well-posed the regularized problem is. **Margin γ:** how far *every* test point sits from the decision boundary. **Failure probability δ.**
- **Samples (Eq. F61):** $M = \tilde{O}\left(\frac{R\,N\,s^7\kappa_{\text{reg}}^4}{\gamma^4\,\delta}\min\{s^2\log^2 D,\ \log^2 m\}\right)$. In words: cost grows with the 7th power of sparsity and 4th power of conditioning, and inversely with the 4th power of the margin.
- The headline "polylog qubits, $\tilde{O}(N)$ samples" holds only when s, κ_{reg}, $1/\gamma$, $1/\delta$ are all polylogarithmic (Eq. F62). A reusable model's qubit count (Eq. F60) has a term growing like s/γ^2.

Data and Methods

- **CFPB Consumer Complaint Database** [21] (Jan–Feb 2026): classify complaint narratives into four products, chronological split. 13,459 training complaints, 50,000 TF-IDF features, 1,563,584 nonzeros; 10 binary tasks (the theorem is binary).
- **UCI Bank Marketing** [22]: 45,211 rows, 51 encoded features, 5 CV folds (dense tabular contrast). **PaySim** [23]: 6,362,620 transactions, analytic only.
- The paper's conventions throughout (unweighted ridge, no intercept, $\|X\| = 1$, labels ± 1). Quantum building blocks were simulated with the authors' reference code [3], corrected where it differed from the theorem. Version-controlled, append-only experiment ledger, regression tests, fixed seeds.

Finding 1: Real Financial Data Fails the Sparsity Condition

[measured inputs; calculated cost]

Dataset	**Column sparsity s**	s/N	κ_{reg}	**Margin γ (10th pct.)**	$\log_{10}$ **passes (F61)**	$\log_{10}$ **if** $s = 1$
CFPB pairs (6)	2,916–5,992	0.58–0.71	5.4–16.5	0.008–0.11	36.7–40.7	11.1–14.7
CFPB one-vs-rest (4)	8,776	0.65	18.7	0.007–0.13	40.9–42.9	13.3–15.3
Bank (5 folds)	36,169	1.00	40.8	0.13–0.15	44.8–44.9	12.2–12.3
PaySim (analytic)	6,362,620	1.00	–	–	s^7 alone: 47.6	–

- **Why:** word frequencies follow a power law, so common words appear in most documents; tabular numeric columns are nonzero in every row. Dropping words in more than 1% of documents still leaves $s \approx 520$–770 and discards 56–60% of the data.
- **Conditioning grows with data size** ($\kappa_{\text{reg}} \propto \sqrt{N}$ at fixed regularization), and the closest test points sit at $\gamma \approx 10^{-5}$.
- The theorem's other preconditions hold (non-trivial fit, sparse test rows). One prediction needs about 52 qubits (A9), but the reusable-model term (F60) is 10^{12}–10^{18}.

Finding 2: The Proof Is Loose, but Not Loose Enough

[simulated: expected-unitary model, ideal uniform sparsity, $N \leq 128$, $s \leq 32$]

Samples M^* each building block needs to reach error ε^*, fitted as $M^* \propto s^\alpha$:

Building block	**Proof exponent**	**Measured,** $N = 64$	**Measured,** $N = 128$
Index oracle	3	2.19–2.20	2.21–2.23
Block encoding (amplified, A/2), raw	5	3.13–3.15	3.25–3.27
Block encoding, log factors removed	5	2.67–2.76	2.79–2.87
Error dependence $\varepsilon^{-\beta}$	$\beta = 1$	1.01–1.02	–

- The proof overcharges by about two powers of s, from splitting its error budget across every oracle call. Exponents are stable across matrix size (largest change 0.07); read them to ± 0.2. Small sizes only: this does not show the asymptotic bound is loose.
- **Found along the way:** `pyqsp` 0.2.0 [6] returns sign polynomials exceeding 1 at $s \geq 16$, and the reference code then silently produces wrong circuit settings (errors up to about 2 on a ± 1 scale). The reference's fixed polynomial degree is also inaccurate from $s = 8$. Both were replaced with a verified construction; a standalone reproduction is in the repository, not yet reported upstream. The paper's real-data plots do not use circuit simulation and are unaffected.

Finding 3: Corrected Cost Is Still Astronomical

[estimated: extrapolated from small-size simulation; orders of magnitude only]

Replacing the proof's s^5 factor with the measured $s^{3.15}$–$s^{3.27}$ (other factors at proof values):

Dataset	$\log_{10}$ **passes, proof**	$\log_{10}$ **passes, corrected**	**Change (decades)**	**Samples per query** ($\varepsilon = 10^{-2}$)
CFPB pairs	36.7–40.7	30.3–34.3	-6.0 to -7.0	1.9×10^{23}–3.6×10^{24}
CFPB one-vs-rest	40.9–42.9	33.6–36.1	-6.8 to -7.3	6.2×10^{24}–1.3×10^{25}
Bank	44.8–44.9	36.4–37.1	-7.9 to -8.4	5.3×10^{26}–1.3×10^{27}

- **Illustration:** at one billion samples per second, a single query to the CFPB data would take about 6 million to 400 million years.
- **Caveats:** 2–3 decades of extrapolation in s; size dependence taken from the theorem; κ, γ, δ factors at proof values.

Finding 4: QOS vs QROM, the Standard Way to Load Stored Data

[estimated: back-of-envelope; units differ, compare orders of magnitude only]

CFPB one-vs-rest matrix (s = 8,776). QROM uses dense encoding with sequential bit packets [8, 9], building on SelectSwap [10, 11], with 1,024 borrowed qubits.

	QROM	**QOS (corrected estimate)**
Cost per query	$\approx 3.3 \times 10^{10}$ Toffoli gates	$\approx 6 \times 10^{24}$–10^{25} samples
Quantum memory	$\approx$ 1,080 qubits (mostly borrowed)	$\approx$ 52 logical qubits (A9)
Data stored?	Yes: 9–13 MB table	Re-streamed $\approx 4 \times 10^{18}$ times per query, plus an 8.2 MB featurizer

- QOS's selling point is not storing the data, but on a finite dataset the data must be stored to be re-streamed. Banks retain data anyway, so QROM-style loading is the realistic route for stored data.
- The amplification step is counted identically for both. Caveats: different units, per query only, QROM figures are lower bounds, fault-tolerance overheads excluded.

Finding 5: Classical Streaming Learners Need Only Kilobytes

[measured: validation selection, frozen configurations confirmed on held-out test data]

Smallest learner memory k^* at which a single-pass learner (feature hashing [18], sparse random projection [19], AWM-Sketch [20]) reaches 95% or 98% of full ridge accuracy; two featurizers, budgets 4–512 KiB, 3 seeds, bootstrap CIs.

Task	k^*_{95} validation	k^*_{95} test	k^*_{98} test
Credit card vs checking (hardest pair)	16 KiB	16 KiB	16–32 KiB
Checking vs mortgage (easy pair)	4–16 KiB	4–16 KiB	16–32 KiB
Mortgage vs rest (imbalanced)	16 KiB	16–32 KiB	32–128 KiB; not reached in 2 of 6 groups

- 16 KiB is about 2,000 stored numbers. Every task reaches 95% on test by 32 KiB, mostly in one pass. Mortgage-vs-rest with the causal featurizer plateaus at about 97.9% for two methods.
- The frozen TF-IDF featurizer (8.2 MB) is about 500 times the learner, and any learner, quantum included, needs it; a causal featurizer cuts the total to about 64–80 KB. A full-dimensional control reached $\geq$99.3% of ridge, so shortfalls come from memory, not optimization.

Scope: What Was and Was Not Tested

- **Tested:** three public datasets (one analytically); the theorem's conditions with the paper's own formulas; QOS building blocks in simulation at small sizes; a QROM cost comparison; tuned classical streaming baselines on three CFPB tasks with held-out test confirmation.
- **Not tested:** internal RBC data; end-to-end scaling in κ, γ, δ; the reusable shadow readout and full solver (calibrated only at $N = 4$); building blocks beyond $N = 128$, $s = 32$ or with non-uniform sparsity; QOS's dimension-reduction and drifting-data theorems; classical streaming on Bank or PaySim.
- **Not shown:** that Theorem F.13 is wrong (all results are consistent with it), or anything about data with bounded row *and* column sparsity.
- The end-to-end simulation was paused because it needed 2–4 weeks of engineering and could not change the result on these datasets. It becomes worthwhile if a candidate dataset passes the sparsity check (see Next Steps). Decision record: `docs/decisions/phase_b_cancelled.md`.

Candidate Bank Data and Problems

QOS can only help where the data (i) has bounded sparsity in rows *and* columns, and (ii) cannot practically be stored or revisited. The table lists bank data worth checking against those conditions, and the condition most likely to fail. Each can be screened in days with the existing audit tool, given data access.

Bank data	Problem it could address	Why it might fit	Check first
Real-time payment and card-authorization streams	Fraud and anomaly scoring on high-volume streams	Very high volume, not retained at full detail; behaviour drifts (the paper's drifting-data theorem targets this)	Popular merchants and accounts make columns dense
Account–counterparty and interbank payment networks	Network risk, liquidity propagation (sparse linear systems, the paper's Theorem 1)	Each account has few counterparties, so rows are sparse	Hub institutions or merchants; networks may fit in classical memory
High-cardinality categorical crosses (merchant × location × device)	Fraud and credit models on feature crosses	Each record activates a small fixed number of features	Common categories dominate columns; hashing already solves memory classically
Market-data and order-book tick streams	Streaming dimension reduction of risk factors	Extreme rates, not stored at tick level	Numeric features are dense ($s = N$, as in Bank)
Complaint, call and chat text (tested: CFPB)	Routing and topic monitoring	Sparse rows	Fails: common words make columns dense

A useful sign of a good candidate: teams currently *discard, subsample or truncate* the data because of memory, not convenience. That matches the project's guiding question: problems that are abandoned or approximated classically.

Alternative Methods Worth Testing

Method	What it is	Why consider it for RBC	Main risk
Fault-tolerant data-loading estimates (QROM / QRAM)	Gate and qubit cost of loading stored bank data [8, 12]	Realistic route for stored data; rapidly improving; natural Xanadu collaboration; no hardware needed	Loading alone is not an advantage; needs a downstream algorithm that pays off
Structured state preparation	Loading smooth distributions cheaply (e.g. Grover–Rudolph [13], learned loaders [14])	Feeds risk-estimation pipelines without loading raw data	Depends on smoothness; QAE-style risk was deprioritized
Quantum kernels and feature maps	Encode inputs into quantum states for similarity-based learning [15]	Small data, so no large loading cost; open research question	No established advantage; kernel concentration [16]
QOS dimension-reduction and drifting-data variants	Other theorems in the same paper [1]	Drift is common in bank data; cheap to audit with the existing tool	Likely the same sparsity requirement
Quantum-inspired classical algorithms	Sampling-based classical analogues of quantum ML [17]	Run on classical hardware today; the right baseline for any QML claim	May not beat standard sketching
Classical streaming sketches (tested)	Hashing, sparse projection, AWM-Sketch	Measured to work at 16–32 KiB; deployable now on high-volume streams	Not quantum

Next Steps

Rough effort estimates; items 1–3 need no new infrastructure.

1. **Positive control (1–2 days).** Run the audit on synthetic data built to satisfy the conditions, to confirm the method recognises a favourable case and not only unfavourable ones.
2. **Audit the other QOS theorems (a few days).** Apply the existing tool to the dimension-reduction and drifting-data results.
3. **Screen one candidate bank dataset (1–2 weeks, incl. access).** Pick from Candidate Bank Data, e.g. payment-network or feature-cross data; measure row and column sparsity, conditioning and margins first. Discussions with Risk and data owners about data that is discarded or truncated for memory reasons would guide the choice.
4. **Resume end-to-end simulation only if step 3 passes (2–4 weeks).** Measure the untested κ, γ, δ scaling on that data.
5. **Fault-tolerant data-loading estimates (2–4 weeks).** Extend the QROM comparison into a resource estimate for a bank workload, a natural fit for a Xanadu conversation.
6. **Package the audit as a reusable tool (about 1 week)** for evaluating future quantum-ML claims against bank data.
7. **Housekeeping.** Report the `pyqsp` issue upstream (with approval); optionally extend classical streaming runs to Bank and PaySim.

Anticipated Questions

The paper is by Caltech and Google researchers. Is it wrong? No. It is a correct worst-case guarantee under stated conditions; these datasets don't meet them, and the cost stays prohibitive after correcting for the proof's looseness.

Was anything run on a quantum computer? No. Building blocks were simulated classically, the standard way to test algorithmic scaling; current hardware cannot run QOS at meaningful sizes.

Would a bigger quantum computer fix this? No. The bottleneck is samples (about 10^{24} per query), not qubits.

What about "50 qubits vs millions of classical bits"? The 50 qubits describe one prediction and omit the dominant costs. Classical learners need 16–32 KiB here.

So is QOS a dead end for RBC? Not established. It is unsuited to the data types tested. Data with bounded row and column sparsity, or genuinely unstorable streams, is untested; Next Steps describes how to check quickly.

Reproducibility

Repository: `https://github.com/THAKKAR82/qos-rbc-experiments` [2], tag `v1-qos-findings`. Index of all results: `results/FINAL_SUMMARY.md`. Reference implementation [3] at commit `a3fe8d6`. Pinned environments (`requirements.txt`; `experiments/qos_tightness/requirements-lock.txt`), append-only ledger (`results/experiment_log.csv`), hashed run manifests and regression tests. Walkthrough for technical and non-technical audiences: `notebooks/08_qos_demo.ipynb`.

Finding	**Main repository location**
1. Applicability audit	`docs/audit_step1_summary.md`, `results/audit/`
2. Tightness study	`experiments/qos_tightness/`, `results/tightness/`
3. Corrected cost	`docs/empirical_f61.md`, `results/audit/empirical_f61/`
4. QROM vs QOS	`docs/qrom_vs_qos_estimate.md`, `results/audit/qrom_vs_qos/`
5. Classical streaming	`results/classical_streaming/`
Scope decision	`docs/decisions/phase_b_cancelled.md`

Glossary

Oracle: quantum access to all data at once. **Sketch:** a compact summary built from a stream. **Sparsity:** most nonzeros in any row or column. **Condition number:** how sensitive a linear problem is; larger is harder. **Margin:** how clearly a test point sits on one side of the decision boundary. **Pass:** one read through the training data. **Logical qubit:** an error-corrected qubit. **QROM:** standard loading of a stored table into a quantum computer. **Toffoli gate:** the costly gate dominating fault-tolerant circuits. **Featurizer:** turns raw text into numbers. **KiB:** 1,024 bytes; 16 KiB holds about 2,000 numbers.

References

[1] H. Zhao, A. Zlokapa, H. Neven, R. Babbush, J. Preskill, J. R. McClean, and H.-Y. Huang, "Exponential quantum advantage in processing massive classical data," arXiv preprint arXiv:2604.07639v2, 2026. Available: arXiv:2604.07639.

[2] D. Thakkar, "qos-rbc-experiments: Evaluating Quantum Oracle Sketching on financial data," GitHub repository, tag `v1-qos-findings`, 2026. Available: github.com/THAKKAR82/qos-rbc-experiments.

[3] H. Zhao *et al.*, "quantum-oracle-sketching," reference implementation, GitHub repository, commit `a3fe8d6`. Available: github.com/haimengzhao/quantum-oracle-sketching.

[4] S. Aaronson, "Read the fine print," *Nature Physics*, vol. 11, pp. 291–293, 2015. doi: 10.1038/nphys3272.

[5] A. Gilyén, Y. Su, G. H. Low, and N. Wiebe, "Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics," in *Proceedings of the 51st ACM Symposium on Theory of Computing*, pp. 193–204, 2019. arXiv: 1806.01838.

[6] J. M. Martyn, Z. M. Rossi, A. K. Tan, and I. L. Chuang, "Grand unification of quantum algorithms," *PRX Quantum*, vol. 2, 040203, 2021. doi: 10.1103/PRXQuantum.2.040203. Software: `pyqsp`.

[7] H.-Y. Huang, R. Kueng, and J. Preskill, "Predicting many properties of a quantum system from very few measurements," *Nature Physics*, vol. 16, pp. 1050–1057, 2020. doi: 10.1038/s41567-020-0932-7.

[8] D. Motlagh, "Halving the cost of QROM again via dense encoding," arXiv preprint arXiv:2610.02321, 2026.

[9] D. Motlagh and M. Pocrnic, "Halving the cost of QROM," arXiv preprint arXiv:2605.20334, 2026.

[10] D. W. Berry, C. Gidney, M. Motta, J. R. McClean, and R. Babbush, "Qubitization of arbitrary basis quantum chemistry leveraging sparsity and low rank factorization," *Quantum*, vol. 3, p. 208, 2019. arXiv: 1902.02134.

[11] G. H. Low, V. Kliuchnikov, and L. Schaeffer, "Trading T gates for dirty qubits in state preparation and unitary synthesis," *Quantum*, vol. 8, p. 1375, 2024. arXiv: 1812.00954.

[12] G. Alonso-Linaje, U. Azad, J. Soni, J. Smalley, L. Lapworth, and J. M. Arrazola, "Quantum compilation framework for data loading," arXiv preprint arXiv:2512.05183, 2025.

[13] L. Grover and T. Rudolph, "Creating superpositions that correspond to efficiently integrable probability distributions," arXiv preprint quant-ph/0208112, 2002.

[14] C. Zoufal, A. Lucchi, and S. Woerner, "Quantum Generative Adversarial Networks for learning and loading random distributions," *npj Quantum Information*, vol. 5, 103, 2019. arXiv: 1904.00043.

[15] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019. doi: 10.1038/s41586-019-0980-2.

[16] S. Thanasilp, S. Wang, M. Cerezo, and Z. Holmes, "Exponential concentration in quantum kernel methods," *Nature Communications*, vol. 15, 5200, 2024. doi: 10.1038/s41467-024-49287-w.

[17] E. Tang, "A quantum-inspired classical algorithm for recommendation systems," in *Proceedings of the 51st ACM Symposium on Theory of Computing*, pp. 217–228, 2019. arXiv: 1807.04271.

[18] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proceedings of the 26th International Conference on Machine Learning*, 2009. arXiv: 0902.2206.

[19] D. M. Kane and J. Nelson, "Sparser Johnson–Lindenstrauss transforms," *Journal of the ACM*, vol. 61, no. 1, 2014. arXiv: 1012.1577.

[20] K. S. Tai, V. Sharan, P. Bailis, and G. Valiant, "Sketching linear classifiers over data streams," in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018. arXiv: 1711.02305.

[21] Consumer Financial Protection Bureau, "Consumer Complaint Database," export of complaints received January–February 2026. Available: consumerfinance.gov/data-research/consumer-complaints.

[22] S. Moro, P. Cortez, and P. Rita, "A data-driven approach to predict the success of bank telemarketing," *Decision Support Systems*, vol. 62, pp. 22–31, 2014. doi: 10.1016/j.dss.2014.03.001.

[23] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proceedings of the 28th European Modeling and Simulation Symposium (EMSS)*, 2016.